

Viva Preparation

Likely questions with model answers based on this code base. File paths are relative to `src/LearnHub/` unless they start with `tests/`, `docs/`, `.github/`, `FirestoreLanding/` or `scripts/`. Every member should be able to answer the architecture, database, hosting and security questions, not only those in their own area.

1. Project and requirements

Q1. What problem does LearnHub solve? Computing students often study from scattered material with no order and no feedback. LearnHub gives them structured courses (lessons and quizzes in a fixed route), explanations after every quiz and a dashboard that shows progress. Administrators get one protected area to manage courses, files, quizzes, users and enrolments.

Q2. Who are the users and what can each do? Guests browse, search and filter the catalogue, open course details and preview lessons, register, log in and send contact messages. Students additionally enrol, study lessons, mark them complete, take quizzes, review results and edit their profile. Administrators manage every entity in `/Admin` and see platform statistics.

Q3. What was deliberately left out, and why? Payments, certificates, chat, forums, an instructor role, email delivery and multiple languages. They would add external services or large features without improving the core learning journey the brief assesses. They are recorded as future enhancements so the scope stayed under control.

Q4. How do you prove every requirement was met? `docs/REQUIREMENTS_CHECKLIST.md` gives each requirement an ID, and `docs/REQUIREMENT_TRACEABILITY.md` maps each ID to the implementing files and to test or screenshot evidence.

2. Architecture and MVC

Q5. Describe the architecture. One ASP.NET Core MVC application in layers: controllers handle HTTP concerns (binding, `ModelState`, redirects and authorisation attributes); services in `Services/` hold business rules and queries; EF Core's `ApplicationContext` talks to the database. Views only display view models. We avoided extra projects or patterns such as CQRS because they would add complexity without benefit at this size.

Q6. Walk through what happens when a student opens a course page. Routing maps `/Courses/Details/5` to `CoursesController.Details`. Middleware has already applied security headers, authentication and authorisation. The action asks `ICourseCatalogService` for a `CourseDetailsViewModel`, which runs an EF Core query (published course, route items, enrolment and progress for this user). The controller returns the view, which Razor renders with the layout.

Q7. Why do services return view models or `OperationResult` instead of entities? Views then receive exactly the data they need, queries select only those columns, and controllers can react to outcomes (success, not found, validation failure) without knowing EF Core. `Services/OperationResult.cs` makes those outcomes explicit.

Q8. How are services wired up? Dependency injection in

`Infrastructure/ServiceCollectionExtensions.cs`: services are scoped (one per request, like the `DbContext`); `FileStorageService` and `TimeProvider.System` are singletons. Controllers receive interfaces through constructors, which also lets tests replace them.

Q9. Why is `TimeProvider` injected instead of calling `DateTime.UtcNow`? Services that record times (enrolments, attempts, completions) can be tested with a fixed clock, and the seeder can create realistic past activity relative to one "now".

Q10. What are Areas, why use one for the admin, and how is that area protected? An Area groups controllers and views under `/Admin` with its own layout (`Areas/Admin/Views/Shared/_AdminLayout.cshtml`). Every admin controller inherits `Areas/Admin/Controllers/AdminControllerBase.cs`, which carries `[Area("Admin")]` and `[Authorize(Roles = AppRoles.Admin)]`, so a new admin page cannot be added without protection. The protection is layered: the role comes from ASP.NET Core Identity rather than a flag in the database, the layout only renders the admin navigation for that role, services protect the last active administrator, and administrators cannot demote, deactivate or delete their own account. Authorisation tests post to admin actions as a student and as a guest and expect a refusal, so the protection is verified rather than assumed.

3. Razor views and front end

Q11. What are tag helpers? Give examples from the project. Server-side components that turn attributes into HTML. Built-in ones such as `asp-for`, `asp-action` and `asp-validation-for` generate names, URLs, antiforgery tokens and validation attributes. We wrote our own in `Infrastructure/TagHelpers.cs`: `lh-nav` highlights the current navigation link and sets `aria-current="page"`, and `<course-cover>` renders a cover image or a fallback icon.

Q12. How did you make the site look like LearnHub rather than default Bootstrap?

`wwwroot/css/site.css` defines design tokens (ink, paper, signal yellow, eight category line colours, Overpass font, spacing and radii) and restyles components around the interchange idea: category lines, course routes, progress bars with a signal marker, and the home page map. Bootstrap provides the grid, utilities and accessible components.

Q13. How is the home page map built? `Infrastructure/InterchangeMap.cs` builds an SVG with `TagBuilder` (so every value is HTML-encoded): up to eight category lines with 45° bends meeting at the hub, each line a link to the filtered catalogue with an accessible name. The CSS animates the lines drawing in only when the user has not asked for reduced motion; the resting style is the finished map.

Q14. What does JavaScript do, and does the site work without it? `wwwroot/js/site.js` adds progressive enhancements: local time formatting, character counters, the password strength meter, image previews, the quiz answered counter and unanswered warning, confirmation prompts, auto-submitting filters, type-specific resource fields and the answer option editor. Every rule is enforced on the server, so forms still work and stay safe without JavaScript.

Q15. Why are Bootstrap and jQuery stored in `wwwroot/lib` instead of loaded from a CDN? The Content Security Policy allows scripts, styles and fonts only from our own origin (`'self'`). Loading from a CDN would require weakening the policy and would make the site depend on a third party.

Q16. How did you make the interface responsive and accessible? Mobile-first CSS with Bootstrap breakpoints: the navbar collapses, the admin sidebar becomes off-canvas below 992 px, admin tables become labelled stacked records below 768 px, and course grids change column counts. Accessibility: semantic landmarks, skip links, labels and hints linked with `aria-describedby`, visible focus styles, `fieldset` and `legend` for quiz questions, and reduced-motion support. Playwright tests run at three sizes, and axe-core reports no WCAG 2.2 A/AA violations.

4. Validation

Q17. How does validation work on the client and on the server? View models carry `DataAnnotations` such as `[Required]`, `[StringLength]`, `[EmailAddress]`, `[RegularExpression]` and `[Compare]` (see `ViewModels/Account/AccountViewModels.cs`). Tag helpers turn them into `data-val-*` attributes that jQuery Validation Unobtrusive enforces in the browser. On submit, model binding validates again and controllers check `ModelState.IsValid` before calling services, because client checks can be bypassed.

Q18. Did you write custom validation attributes? Yes. `Infrastructure/MustBeTrueAttribute.cs` requires the terms checkbox, and `Infrastructure/FileValidationAttributes.cs` checks upload sizes and extensions. Both implement `IClientModelValidator`, so the same rule also runs in the browser through `wwwroot/js/validation.js`.

Q19. Which rules cannot be expressed with attributes? Business rules that need the database or several fields: unique category names, exactly one correct answer among two to six options, a quiz needing a question before publishing, enrolment only in published courses, and protection of the last administrator. Services check them and return `OperationResult` failures that controllers add to `ModelState`.

Q20. How do you keep form limits and database column sizes consistent? `Models/FieldLengths.cs` is the single source of truth: EF Core configurations use it for `HasMaxLength` and view models use it in `[StringLength]`, so a form can never accept more than the column holds.

5. Database design

Q21. Explain the main entities and relationships. A category has many courses. A course has many learning resources, quizzes and enrolments. A quiz has questions, a question has answer options. A student has enrolments, resource completions and quiz attempts; an attempt has one quiz answer per question. Contact messages stand alone. That is eleven application tables plus the seven ASP.NET Core Identity tables. The full diagram is in `docs/ERD.md` and the decisions behind it are in `docs/DATABASE.md`.

Q22. Which constraints protect data integrity? Primary and foreign keys; required columns with maximum lengths; unique indexes on category name, `(UserId, CourseId)` for enrolments, `(UserId, LearningResourceId)` for completions and `(QuizAttemptId, QuestionId)` for answers; and check constraints such as `DurationMinutes > 0` and `ScorePercent BETWEEN 0 AND 100` (`Data/Configurations/`).

Q23. Why does deleting a category restrict, while deleting a course cascades? A category is a label shared by many courses; deleting it by mistake must not wipe courses, so the delete is blocked while courses exist. A course owns its lessons, quizzes and enrolments, which are meaningless without it, and the confirmation page shows exactly what will be removed.

Q24. Why are `QuizAnswer → Question` and `QuizAnswer → AnswerOption` set to Restrict? Because a quiz can be edited after students have taken it. A cascade from a question or an option down to the stored answers would let one edit silently erase a student's answer history, so the foreign keys restrict instead. When a quiz, question or option really has to go, the services delete the dependent answers explicitly first, inside one transaction, so the removal is deliberate and ordered.

Q25. Why do quiz attempts store the score instead of recalculating it? Administrators can edit a quiz after students took it. Storing `CorrectCount`, `QuestionCount`, `ScorePercent`, `Passed` and `IsCorrect` per answer keeps every student's history truthful.

Q26. And why is course progress not stored? Progress depends on the current number of lessons and published quizzes. Calculating it from completions and passed attempts means adding a lesson correctly lowers everyone's percentage, with no stale column to update.

Q27. How are dates handled? All `DateTime` values are stored as UTC through a value converter configured in `Data/ApplicationDbContext.cs`, and `CreatedAt` and `UpdatedAt` are set automatically in `SaveChanges`. The browser converts them to local time.

6. Entity Framework Core

Q28. Why is SQLite the only provider, and what would a second provider cost?

`Data/DatabaseServiceCollectionExtensions.cs` calls `UseSqlite` unconditionally, so there is one context, one migration set and one schema to reason about in development, in tests and in production. An earlier version of the project kept SQLite for development and SQL Server for production, which meant an abstract base context, two derived contexts, two migrations folders and a test matrix that ran everything twice. Removing it deleted a whole class of dialect differences — and the bugs they cause — at the cost of accepting SQLite's single-writer, file-based model, which is a reasonable trade for a single-instance course project.

Q29. How are migrations created and applied? `dotnet ef migrations add InitialCreate --project src/LearnHub --output-dir Data/Migrations` uses `Data/DesignTimeDbContextFactory.cs`, the single design-time factory, because the application's own start-up code is not what `dotnet ef` runs. One migration exists — `20260914100832_InitialCreate` — and it creates all eighteen tables: eleven application tables plus the seven Identity tables. `.github/workflows/ef-migrations.yml` regenerates and commits a migration when the model changes, and at start-up `Data/Seed/DatabaseInitializer.cs` applies any pending migration before the application serves a request.

Q30. You found a query that SQLite could not translate. What happened? The admin course details query projected nested collections with `Take`, which translates to a correlated subquery and, on some providers, to SQL `APPLY`. SQLite has no `APPLY`, so the page returned 500 in the tests. We rewrote it as separate flat queries — the course, then its recent enrolments — which SQLite executes happily (`Services/CourseManagementService.cs`).

Q31. Why use `AsNoTracking`, `ExecuteDeleteAsync` and projections? Read-only queries use `AsNoTracking` and project to view models, so EF Core neither tracks entities nor loads unneeded columns. `ExecuteDeleteAsync` deletes rows in the database with one statement, which avoids loading whole object graphs and avoids the relationship errors we hit when deleting tracked dependants.

Q32. How do transactions work here? `Data/TransactionExtensions.cs` runs the unit of work inside `Database.CreateExecutionStrategy().ExecuteAsync`, with the transaction started and committed inside that delegate. SQLite's default strategy simply executes once, so the helper costs nothing today; the point is that the whole unit is retried together the moment a retrying strategy is configured, and the calling code does not have to change if the provider ever does. The seeding path uses the same helper, which is why a slow or interrupted first start cannot leave half a catalogue behind.

Q33. Is LINQ safe from SQL injection? Yes: EF Core sends values as parameters. The catalogue search uses `EF.Functions.Like` with a pattern built by `Services/SearchPattern.cs`, which also escapes `%`, `_` and `[` so they are matched literally.

Q34. What happens if two requests enrol the same student at the same time? Both may pass the "already enrolled" check, but the unique `(UserId, CourseId)` index rejects the second insert. `Services/EnrollmentService.cs` catches the `DbUpdateException`, clears the change tracker and shows "You are already enrolled in this course." instead of an error page.

7. Authentication and authorisation

Q35. How are passwords stored, and why are the demo passwords not in Git? ASP.NET Core Identity stores a salted PBKDF2 hash with many iterations, never the password. The policy (8–100 characters with upper-case, lower-case and a digit) is applied in Identity options and in the registration view model. No password is ever committed: in Development, `DevelopmentSeedPasswords` generates strong random passwords and writes them to `App_Data/demo-credentials.json`, which `.gitignore` excludes, so a fresh clone can sign in as the DEMO ONLY administrator and student without a secret in the repository. In production the passwords must come from the `Seed__AdminPassword` and `Seed__DemoStudentPassword` variables, and `.env.example` documents every variable with a placeholder only.

Q36. How are roles assigned? Registration adds the `Student` role. The first administrator is created at start-up from configuration (`Seed:AdminEmail`, `Seed:AdminPassword`) if no such account exists; after that, administrators change roles in the admin area.

Q37. Why is the admin password not in `appsettings.json`? Anything in Git can leak, and a repository is copied, forked and archived. Locally the password is set with `dotnet user-secrets` or generated into the git-ignored `App_Data/demo-credentials.json`; in production it is a hosting variable (`Seed__AdminPassword`) that exists only in the platform's environment. With no password set, the account is simply not created, so a misconfigured deployment cannot end up with a weak default administrator.

Q38. What protects against brute-force login attempts? Identity logout (five failed attempts lock the account for 15 minutes), a generic error message that does not reveal whether an email exists, and a rate limit of 10 form submissions per minute per IP on login, registration, password change and contact.

Q39. How does deactivating a user work, and how fast does it take effect?

`Services/UserManagementService.cs` sets `LockoutEnd` to the maximum date and updates the security stamp. Identity refuses future sign-ins, and because the cookie's security stamp is re-validated every five minutes, an existing session ends within that time. Administrators cannot deactivate, demote or delete themselves or the last active administrator.

Q40. How does the app redirect users after login without an open redirect?

`AccountController.RedirectToLocal` follows `returnUrl` only when `Url.IsLocalUrl` accepts it;

otherwise administrators go to `/Admin` and students to their dashboard. A test tries an external URL and expects the dashboard.

8. Security

Q41. How do you prevent CSRF? `ValidateAntiForgeryTokenAttribute` is registered globally, so every unsafe HTTP method needs a valid token, which the form tag helper adds automatically. All state changes are POST forms. A test posts to login without a token and expects 400.

Q42. How do you prevent XSS, especially in lesson content written by admins? Razor encodes all output and the views never use `Html.Raw` on user input. Lesson text goes through `Services/Content/LessonContentRenderer.cs`, which encodes everything first and then applies a fixed allow-list of formatting. A strict CSP blocks inline scripts as a second layer.

Q43. What is IDOR and where did you prevent it? Insecure direct object reference: changing an id in a URL to reach someone else's data. `QuizService` returns a result only for the attempt's owner (others get 404), lessons and files require enrolment unless they are previews, and admin actions require the Admin role. Tests cover each case.

Q44. Why not bind the entity directly in forms? Overposting: a user could add fields such as a role flag or `IsPublished` to the request. Forms bind to view models that contain only editable fields, and services copy those values onto entities.

Q45. How are file uploads secured? `Services/Storage/FileStorageService.cs` checks the declared size, the extension allow-list and the file signature (the first bytes must be JPEG, PNG, WebP or PDF), saves under a random name outside `wwwroot`, and enforces the size again on the bytes actually written. Resource files are streamed by `ResourcesController.Open` after the access check.

Q46. How are embedded videos kept safe? `Services/Content/VideoEmbedParser.cs` accepts only known YouTube and Vimeo URL formats, extracts the video id and builds a `youtube-nocookie.com` or `player.vimeo.com` embed URL itself. The CSP `frame-src` allows only those two players.

Q47. Which security headers are sent? `Infrastructure/SecurityHeadersMiddleware.cs` sends a Content Security Policy, `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy`, `Permissions-Policy` and `Cross-Origin-Opener-Policy`; production also uses HTTPS redirection and HSTS.

Q48. How does the application reach its database, and is there a database password to protect? There is no database server to authenticate to. SQLite is a file, so the application opens `Data Source=/data/learnhub.db`, a path supplied by the `DATABASE_CONNECTION_STRING` variable; the only credential in the system is the SQLite file itself, protected by the container's file system and by the platform's access control to the service and its volume. That removes an entire class of risk — no connection string with a password, no firewall rule and no network database port to expose — and it is why the volume is the piece that matters: whoever has the file has the data.

Q49. How are production secrets supplied now that there is no cloud key vault? Through hosting variables. `.env.example` documents each one — `DATABASE_CONNECTION_STRING`, `Seed__AdminPassword`, `Seed__DemoStudentPassword`, `DataProtection__KeysPath`, `Storage__RootPath` — with placeholders only, while the real `.env` file is git-ignored, and GitHub Actions generates throw-away credentials per run,

masks them in the log and never stores them. For a project of this size that is proportionate: the secrets live in one place, they are never in the repository, and no code change is needed when the environment changes because ASP.NET Core reads the variables directly.

9. Testing

Q50. What kinds of tests did you write? Unit tests (grading, progress, validation, content safety, file signatures), service tests against a real in-memory SQLite database, integration tests that run the whole app in memory with `WebApplicationFactory`, a crawler that follows every link as each role, Playwright browser scenarios at three screen sizes with axe-core accessibility scans, a container job that starts the Docker image the way the host will and smoke tests it, and `scripts/smoke-test.sh` for a running instance. In total 211 .NET test cases and 42 browser checks (14 browser tests × 3 viewports).

Q51. How do integration tests log in and submit forms?

`tests/LearnHub.Tests/Infrastructure/BrowserClientExtensions.cs` gets the page, extracts the antiforgery token from the HTML, posts the form with cookies kept by the client and follows redirects manually, just like a browser.

Q52. How do you test the container and the volume the way the host will run them? The `container` job in `.github/workflows/ci.yml` builds the same `Dockerfile` Railway builds, then runs it with `ASPNETCORE_ENVIRONMENT=Production`, `PORT=8080` and a volume mounted at `/data`. It waits for `/health`, runs the smoke test, asserts from the log that migrations were applied, the demo data was seeded and the administrator was created with no `fail:` or `crit:` lines, and then restarts the container and asserts that migrations were **not** re-applied. That last assertion is the real test of persistence: the SQLite file and the Data Protection key ring only survive the restart because they live on the volume.

Q53. Which bugs did testing find? The SQLite `APPLY` failure, a relationship error when deleting courses, form pages failing behind a TLS-terminating proxy until forwarded headers were configured, a migration that was never regenerated after a model change and broke 126 tests, several layouts scrolling sideways on phones and tablets, a transparent admin sidebar, map labels overlapping, and accessibility issues (unfocusable scroll regions, links without names on phones). All are listed with their fixes in `docs/TESTING.md`.

10. Deployment, hosting and persistence

Q54. Why is the application on Railway while Firebase Hosting serves a separate site? Because Firebase Hosting cannot host it. Firebase Hosting serves static files only: there is no server-side runtime, so it cannot execute ASP.NET Core, and it offers no writable file system for a SQLite database or uploads. The application therefore runs as a Docker container on Railway, and the presentation site stays a static page on Firebase, which links to the application with its "Open Learning System" buttons. That split is deliberate: each host does the one thing it is good at.

Q55. What do the GitHub Actions workflows do? `ci.yml` has three jobs. `build-and-test` restores, builds with `-p:TreatWarningsAsErrors=true` and runs the whole .NET test suite. `container` builds the Docker image and runs it the way Railway does — Production, `PORT=8080`, a volume at `/data` — smoke tests it, checks the log, and restarts it to prove the data persisted. `e2e` runs the Playwright suite against a production instance over HTTPS. `ef-migrations.yml` regenerates and commits the SQLite migration when

the model changes. The old Azure deploy, GitHub Pages and provisioning workflows were removed along with the platform they targeted.

Q56. What does `UsePlatformProxyHeaders` do, and why is it needed? Railway terminates TLS at its edge proxy and forwards plain HTTP inside the platform, describing the original request in `X-Forwarded-Proto` and `X-Forwarded-For`. ASP.NET Core does not read those headers by itself, so `ApplicationBuilderExtensions.UsePlatformProxyHeaders()` enables `ForwardedHeadersMiddleware` for exactly those two headers. It is needed because without it the application believes every request arrived over HTTP: HSTS would never be sent, secure cookies and HTTPS redirection would misbehave, and every request would look like it came from the proxy — which matters because the rate limiter partitions its fixed window by `context.Connection.RemoteIpAddress`, so all users would share one counter and one abuser could lock everybody out. The loopback-only default allow-list is cleared deliberately, because the platform proxy is the only route into the container and arrives from a private-network address; the middleware therefore runs first, before anything that inspects the scheme, the client address or a generated link. `ASPNETCORE_FORWARDEDHEADERS_ENABLED` does not do this job — it was an Azure App Service hosting feature, not part of the framework.

Q57. Why is `PORT` read from the environment? Platforms like Railway choose the port when the container starts and pass it in `PORT`; hard-coding a port would mean the router and the application disagree. `UsePlatformPort()` reads the variable at start-up and binds `http://0.0.0.0:{PORT}` so the container accepts traffic from outside its own network namespace, and falls back to the normal ASP.NET Core URL configuration when `PORT` is absent. The `Dockerfile` sets `PORT=8080` only as a fallback for running the image locally, which is also how the CI container job exercises it.

Q58. Why SQLite, and how is the database persisted on Railway? SQLite keeps the whole relational model in one file with real foreign keys and transactions, and it needs no server, no connection credential and no network hop, which suits a single-instance academic application. The catch is that a container's own file system is disposable, so the file must not live there. The connection string comes from the `DATABASE_CONNECTION_STRING` variable (`Data Source=/data/learnhub.db`), and `/data` is a Railway volume: storage that is mounted into the container and survives restarts, redeploys and image changes. `Data/DatabaseServiceCollectionExtensions.cs` resolves a relative path against the content root and creates its folder, so the same code works with the default `App_Data/learnhub.db` on a developer machine and with the volume path in production.

Q59. What lives on the `/data` volume, and what breaks without it? Three things, and each fails differently. The SQLite file (`/data/learnhub.db`) holds every course, enrolment, attempt and account, so without the volume each redeploy starts from an empty database and re-seeds the demo catalogue. The Data Protection key ring (`DataProtection__KeysPath=/data/keys`) encrypts the authentication cookies, so without it every restart invalidates them and signs every user out — re-running migrations would not fix that. Administrator uploads (`Storage__RootPath=/data/storage`) hold the course thumbnails and resource files, so without it the database rows survive while the files they point at disappear. This is exactly what the `container` job checks when it restarts the container and asserts that migrations were not re-applied.

Q60. How are migrations and seeding run at start-up, and is that safe? `Program.cs` calls `InitializeDatabaseAsync()` after the routes are mapped and before the application starts serving. `Data/Seed/DatabaseInitializer.cs` applies pending migrations when `Database:ApplyMigrationsOnStartup` is true (the default), creates the `Admin` and `Student` roles, creates the administrator from configuration if no such account exists, and then lets `DemoDataSeeder` fill an empty

database with the demonstration catalogue. Seeding runs only when no categories exist, so it can never duplicate or overwrite real data. For this single-instance deployment the approach is safe — one container, and EF Core takes a migration lock so two instances could not migrate at once — and it removes the "forgot to migrate" failure mode that had already broken 126 tests once. The trade-off is that a start-up migration is part of the request path's readiness: it is why the health check is `/health` and why the Railway health check timeout is 300 seconds.

Q61. How is the image built and configured for Railway? The `Dockerfile` is multi-stage:

`mcr.microsoft.com/dotnet/sdk:10.0` restores and publishes the project, and `mcr.microsoft.com/dotnet/aspnet:10.0` carries only the published output, so the SDK and the NuGet cache never reach the running container. `railway.json` selects the Dockerfile builder, sets the health check to `/health` with a 300-second timeout, and restarts the service on failure. There is no separate migration step in the pipeline because the container migrates and seeds itself on start-up.

Q62. What is deployed today? Firebase Hosting is live at <https://learnhub-wapp.web.app>: the landing page, its design system, the Overpass webfont and the eight screenshots are served from `FirebaseLanding/`, and unknown paths return the custom 404 page. The Railway service has **not** been deployed yet — the CLI is installed but not authenticated, so we can honestly say the image, the migrations and the volume behaviour are proven in CI and locally, while the hosted deployment is the next step.

Q63. How does the Firebase site point at the application? The landing page ships with the conventional address `https://learnhub-production.up.railway.app`, because Railway generates the real domain when the service is created. After the first deploy, `scripts/set-app-url.sh <railway-url>` rewrites every call-to-action in `FirebaseLanding/index.html`, and `firebase deploy --only hosting` republishes the site. `firebase.json` sets the public folder to `FirebaseLanding` with caching headers, and `.firebaserc` sets the default project to `learnhub-wapp`.

11. Teamwork and reflection

Q64. How did the team avoid conflicts? Clear ownership per area, short-lived branches, pull requests with green CI and review, small changes to shared files such as `site.css` and the EF Core model, and regenerating migrations instead of hand-merging snapshots.

Q65. What are the main limitations? No email confirmation or password reset by email, no multi-factor authentication, and a single-instance design: the SQLite file, the Data Protection key ring and the uploads share one mounted volume, so scaling out would need a different database and object storage. The hosted Railway deployment has not been made yet, and there is no monitoring or alerting.

Q66. What would you add next? Email confirmation and password reset, MFA for administrators, certificates on course completion, an instructor role, a managed relational database with object storage so the application can run more than one instance, monitoring and alerting for the deployed service, and Dependabot for dependency updates.